ASSIGNMENT NO 05: Design an Assignment to retrieve, verify, and store user credentials using Firebase Authentication, the Google App Engine standard environment, and Google Cloud Data store.

Software Requirements:
Ubuntu 16.04
Python
MySQL

Theory:
Authenticating Users on App Engine Using Firebase
This assignment shows how to retrieve, verify, and store user credentials using Firebase Authentication, the Google App Engine standard environment, and Google Cloud Datastore. The document walks you through a simple note-taking application called Firenotes that stores users' notes in their own personal notebooks. Notebooks are stored per user, and identified by each user's unique Firebase Authentication ID.
The application has the following components:

- **The frontend configures the sign-in user interface and retrieves the Firebase Authentication ID. It also handles authentication state changes and lets users see their notes. FirebaseUI is an open-source, drop-in solution that handles user login, linking multiple providers to one account, recovering passwords, and more. It implements authentication best practices for a smooth and secure sign-in experience.**
- **The backend verifies the user's authentication state and returns user profile information as well as the user's notes.**

The application stores user credentials in Cloud Datastore by using the NDB client library, but you can store the credentials in a database of your choice.

The following diagram shows how the frontend and backend communicate with each other and how user credentials travel from Firebase to the database.
Firenotes is based on the Flask web application framework. The sample app uses Flask because of its simplicity and ease of use, but the concepts and technologies explored are applicable regardless of which framework you use.
Objectives

- **Configure the Firebase Authentication user interface.**
- **Obtain a Firebase ID token and verify it using server-side authentication.**
- **Store user credentials and associated data in Cloud Datastore. Query a database using the NDB client library. Deploy an app to App Engine.**

Costs
This assignment uses billable components of Cloud Platform, including:

Google Cloud Datastore

Use the Pricing Calculator to generate a cost estimate based on your projected usage. New Cloud Platform users might be eligible for a free trial.

Before you begin

1. **Install Git, Python 2.7, and virtualenv. For more information on setting up your Python development environment, such as installing the latest version of Python, refer to Setting Up a Python Development Environment for Google Cloud Platform.**
2. **Select or create a Google Cloud Platform project.**

   Note: If you don't plan to keep the resources you create in this tutorial, create a new project instead of selecting an existing project. After you finish, you can delete the project, removing all resources associated with the project and tutorial.

   Go to the Manage resources page
3. **Install and initialize the Cloud SDK.**

If you have already installed and initialized the SDK to a different project, set the gcloud project to the App Engine project ID you're using for Firenotes. See Managing Cloud SDK Configurations for specific commands to update a project with the gcloud tool.

Cloning the sample app

To download the sample to your local machine:

1. **Clone the sample application repository to your local machine:**

   git clone https://github.com/GoogleCloudPlatform/python-docs-samples.git Alternatively, you can download the sample as a zip file and extract it.
2. **Navigate to the directory that contains the sample code:**
   cd python-docs-samples/appengine/standard/firebase/firenotes

Adding the Firebase Authentication user interface

To configure FirebaseUI and enable identity providers:

1. **Add Firebase to your app by following these steps:**
   1. **Create a Firebase project in the Firebase console.**
      If you don't have an existing Firebase project, click Add project and enter either an existing Google Cloud Platform project name or a new project name.
      If you have an existing Firebase project that you'd like to use, select that project from the console.
   2. **From the project overview page, click Add Firebase to your web app. If your project already has an app, select Add App from the project overview page.**

3. Use the Initialize Firebase section of your project's customized code snippet to fill out the following section of the frontend/main.js file:

// Obtain the following from the "Add Firebase to your web app" dialogue

```
// Initialize Firebase
var config = {

apiKey: "<API_KEY>",
authDomain: "<PROJECT_ID>.firebaseapp.com",
databaseURL: "https://<DATABASE_NAME>.firebaseio.com",
projectId: "<PROJECT_ID>",
storageBucket: "<BUCKET>.appspot.com",
messagingSenderId: "<MESSAGING_SENDER_ID>"
};
```

2. Edit the backend/app.yaml file and enter your Firebase project ID in the environment variables:

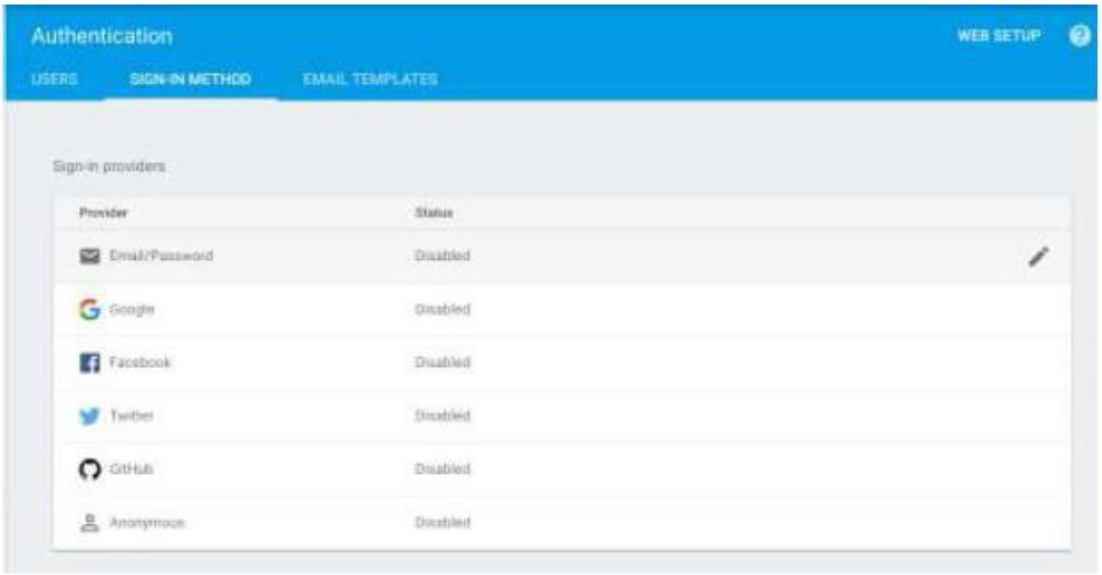
```
runtime: python27
api_version: 1
threadsafe: true
  service: backend

  handlers:
  - url: /.*
  script: main.app

  env_variables:
  # Replace with your Firebase project ID.
  FIREBASE_PROJECT_ID: '<PROJECT_ID>'
```

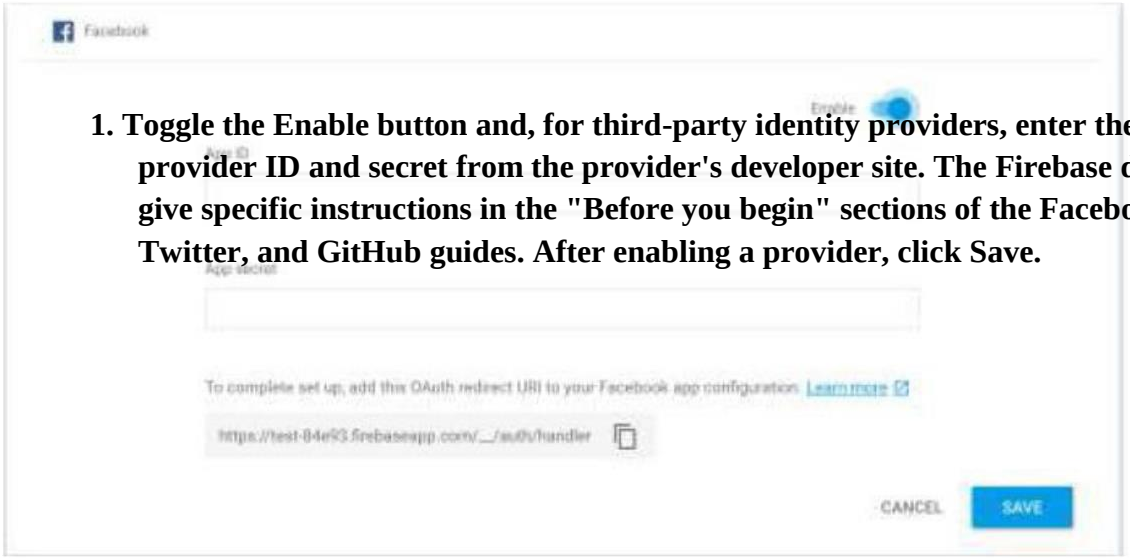
3. In the frontend/main.js file, configure the FirebaseUI login widget by selecting which providers you want to offer your users.

```
// Firebase log-in widget
function configureFirebaseLoginWidget() {
var uiConfig = {
'signInSuccessUrl': '/',
'signInOptions': [
// Leave the lines as is for the providers you want to offer your
users. firebase.auth.GoogleAuthProvider.PROVIDER_ID,
firebase.auth.FacebookAuthProvider.PROVIDER_ID,
firebase.auth.TwitterAuthProvider.PROVIDER_ID,
firebase.auth.GithubAuthProvider.PROVIDER_ID,
firebase.auth.EmailAuthProvider.PROVIDER_ID
],
// Terms of service url
'tosUrl': '<your-tos-url>',
};
```

```
var ui = new
firebaseui.auth.AuthUI(firebase.auth());
ui.start('#firebaseui-auth-container', uiConfig);
}
```

4. **Enable the providers you have chosen to keep in the Firebase console by clicking Authentication > Sign-in method. Then, under Sign-in providers, hover the cursor over a provider and click the pencil icon.**





1. **Toggle the Enable button and, for third-party identity providers, enter the provider ID and secret from the provider's developer site. The Firebase docs give specific instructions in the "Before you begin" sections of the Facebook, Twitter, and GitHub guides. After enabling a provider, click Save.**

2. In the Firebase console, under Authorized Domains, click Add Domain and enter the domain of your app on App Engine in the following format: [PROJECT_ID].appspot.com

Do not include http:// before the domain name.

Installing dependencies
Navigate to the backend directory and complete the application setup:

Mac OS / Linux
Windows

1. **Create an isolated Python environment in a directory external to your project and activate it:**
   virtualenv env
   source env/bin/activate
2. **Navigate to your project directory and install dependencies:**
   cd YOUR _PROJECT
   pip install -t lib -r requirements.txt

In appengine_config.py, the vendor.add() method registers the libraries in the lib directory.

Running the application locally
To run the application locally, use the App Engine local development server:

1. **Add the following URL as the backendHostURL in main.js:**
   http://localhost:8081
2. **Navigate to the root directory of the application. Then, start the development server:**

   dev_appserver.py frontend/app.yaml backend/app.yaml
3. **Visit http://localhost:8080/ in a web browser.**

Authenticating users on the server
Now that you have set up a project and initialized an application for development, you can walk through the code to understand how to retrieve and verify Firebase ID tokens on the server.
Getting an ID token from Firebase

The first step in server-side authentication is retrieving an access token to verify. Authentication requests are handled with the onAuthStateChanged() listener from Firebase:

```
firebase.auth().onAuthStateChanged(function(us
er) { if (user) {
$('#logged-out').hide();
var name = user.displayName;
/* If the provider gives a display name, use the name for
the personal welcome message. Otherwise, use the user's
```

```
email. */ var welcomeName = name ? name : user.email;
user.getToken().then(function(idToken) {

userIdToken = idToken;
/* Now that the user is authenicated, fetch the
notes. */ fetchNotes();
$('#user').text(welcomeName);
$('#logged-in').show();
});
 } else {
$('#logged-in').hide();
$('#logged-out').show();
```

When a} user is signed in, the Firebase getToken() method in the callback returns a Firebase ID token in the form of a JSON Web Token (JWT).

Verifying tokens on the server

After a user signs in, the frontend service fetches any existing notes in the user's notebook through an AJAX GET request. This requires authorization to access the user's data, so the JWT is sent in the Authorization header of the request using the Bearer schema:

```
// Fetch notes from the backend.
function fetchNotes() {
$.ajax(backendHostUrl +
'/notes', {
/* Set header for the XMLHttpRequest to get data from the web
server associated with userIdToken */

headers: {

'Authorization': 'Bearer ' + userIdToken
}
})
```

Before the client can access server data, your server must verify the token is signed by Firebase. You can verify this token using the Google Authentication Library for Python. Use the authentication library's verify_firebase_token function to verify the bearer token and extract the claims:

```
id_token = request.headers['Authorization'].split('
').pop()                    claims                    =
google.oauth2.id_token.verify_firebase_          token(
id_token, HTTP_REQUEST)
if not claims:
```

```
    return 'Unauthorized', 401
```

Each identity provider sends a different set of claims, but each has at least a sub claim with a unique user ID and a claim that provides some profile information, such as name or email, that you can use to personalize the user experience on your app.

Managing user data in Cloud Datastore

After authenticating a user, you need to store their data for it to persist after a signed-in session has ended. The following sections explain how to store a note as a Cloud Datastore entity and segregate entities by user ID.

Creating entities to store user data

You can create an entity in Cloud Datastore by declaring an NDB model class with certain properties such as integers or strings. Cloud Datastore indexes entities by kind; in the case of Firenotes, the kind of each entity is Note. For querying purposes, each Note is stored with a key name, which is the user ID obtained from the sub claim in the previous section.

The following code demonstrates how to set properties of an entity, both with the constructor method for the model class when the entity is created and through assignment of individual properties after creation:

```
    data = request.get_json()
    # Populates note properties according to the model,
    # with the user ID as the key name.

    note = Note(
    parent=ndb.Key(Note, claims['sub']),
    message=data['message'])

    # Some providers do not provide one of these so either can be
    used. note.friendly_id = claims.get('name', claims.get('email',
    'Unknown'))
```

To write the newly created Note to Cloud Datastore, call the put() method on the note object. Retrieving user data

To retrieve user data associated with a particular user ID, use the NDB query() method to search the database for notes in the same entity group. Entities in the same group, or_ancestor path_, share a common key name, which in this case is the user ID.

```
    def query_database(user_id):
    """Fetches all notes associated with user_id.
    Notes are ordered them by date created, with most recent note
    added first."""

    ancestor_key = ndb.Key(Note, user_id)
```

```
query = Note.query(ancestor=ancestor_key).order(-Note.created)
notes = query.fetch()
note_messages = []
for note in notes:
note_messages.append({
'friendly_id': note.friendly_id,
'message': note.message,
'created': note.created
})
return note_messages
```

You can then fetch the query data and display the notes in the client:

```
// Fetch notes from the backend.
function fetchNotes() {
$.ajax(backendHostUrl +
'/notes', {
/* Set header for the XMLHttpRequest to get data from the web
server associated with userIdToken */
headers: {
'Authorization': 'Bearer ' + userIdToken
}
}).then(function(data){
$('#notes-container').empty();
// Iterate over user data to display user's notes from database.
data.forEach(function(note){
$('#notes-container').append($('<p>').text(note.message));

});
});
}
```

Deploying your app

You have successfully integrated Firebase Authentication with your App Engine application. To see your application running in a live production environment:

1. **Change the backend host URL in main.js to https://backend-dot-[PROJECT _ID].appspot.com. Replace [PROJECT_ID] with your project ID.**

2. **Deploy the application using the Cloud SDK command-line interface:**
   gcloud app deploy backend/index.yaml frontend/app.yaml backend/app.yaml
3. **View the application live at https://[PROJECT_ID].appspot.com.**

Note: Cloud Datastore indexes take a few minutes to update, so the application might not be fully functional immediately after deployment.

Cleaning up

To avoid incurring charges to your Google Cloud Platform account for the resources used in this tutorial, delete your App Engine project:

Deleting the project

The easiest way to eliminate billing is to delete the project you created for the tutorial.

1. **To delete the project:**
   Caution: Deleting a project has the following effects:
   - **Everything in the project is deleted. If you used an existing project for this tutorial, when you delete it, you also delete any other work you've done in the project.**
   - **Custom project IDs are lost. When you created this project, you might have created a custom project ID that you want to use in the future. To preserve the URLs that use the project ID, such as an appspot.com URL, delete selected resources inside the project instead of deleting the whole project.**

If you plan to explore multiple tutorials and quickstarts, reusing projects can help you avoid exceeding project quota limits.

2. **In the GCP Console, go to the Projects page.**
   Go to the Projects page
3. **In the project list, select the project you want to delete and click Delete delete.**
4. **In the dialog, type the project ID, and then click Shut down to delete the project.**

Conclusion :

Here we learnt to retrieve, verify, and store user credentials using Firebase Authentication, the Google App Engine standard environment, and Google Cloud Data store.